

HEINRICH-HEINE-UNIVERSITÄT DÜSSELDORF

Bachelor Thesis

Implementierung und Evaluation der Reinforcement-Learning-Methoden Self-Play und Liga-Training in der MicroRTS-Umgebung

Name!!

Degree:	B.Sc. Computer Science
Supervisor:	Professor Dr. Paul Swoboda
Co-Supervisor:	Dr. Peter Arndt
Faculty:	Mathematisch-Naturwissenschaftliche Fakultät

XXXXXXXXXXXX

Abstract

Diese Arbeit verbessert einen bestehenden Agenten für das Echtzeit-Strategiespiel (RTS) MicroRTS. RTS-Spiele gelten als anspruchsvolle Domäne für Deep Reinforcement Learning (DRL), da Agenten mit großen kombinatorischen Aktionsräumen, simultanen Entscheidungen sowie verzögerten und oftmals spärlichen Belohnungen umgehen müssen. Als Basismethode verwenden wir Proximal Policy Optimization (PPO). Bei vielen DRL -Trainingsarten, wie bei reinem PPO, kommt man an einen Punkt, an dem der Agent nicht effizient ausschließlich mit PPO weiter trainiert werden kann, weil ausreichend starke Gegner, gegen die man trainieren kann fehlen oder, weil die vorhandenen Gegner nicht divers genug sind, um eine robuste Policy zu entwickeln. Eine naheliegende und einfache Gegenmaßnahme ist Self-Play (SP), das jedoch in der Praxis instabil sein kann und zu Überanpassung oder zyklischen Strategien führt, wodurch die Leistung gegen externe Gegner sinken kann.

League-Training (LT) stabilisiert SP, indem es eine Population aus Agenten unterschiedlicher Trainingsstände und Rollen aufbaut. Der trainierenden Agent wird gezielt gegen vielfältige Gegner antreten. Unter anderem auch gegen spezialisierte Agenten, die trainiert wurden, Schwachstellen der aktuellen Policy auszunutzen. Wir evaluieren die resultierenden Agenten in Matches gegen verschiedene Gegnertypen und vergleichen LT mit reinem SP sowie unserem Anfangs-Agenten. Der Anfangs-Agent wurde mit einer Mischung aus Behavior Cloning (BC) und PPO Trainiert. Unsere Ergebnisse zeigen, dass LT Robustheit und Generalisierungsfähigkeit des Agenten gegenüber unterschiedlichen Gegnern erhöht.

Contents

1. Einleitung	1
2. Related Work	2
3. Methoden	4
3.1. MicroRTS-Umgebung	4
3.2. Training Pipeline	5
4. Experimente	6
5. Conclusion	7
Erklärung	9

1. Einleitung

Reinforcement Learning (RL) etablierte sich in den vergangenen Jahren als leistungstarker Ansatz für die Entwicklung von Agenten in komplexen Umgebungen. Spielumgebungen wie RTS-Spiele stellen dabei eine besondere Herausforderung dar, mit großen Aktionsräumen, langfristiger Planung und simultanen Entscheidungen. Allerdings bieten sie klar definierte Zustandsräume, Belohnungsstrukturen und wiederholbare Experimente, was sie zu idealen und herausfordernden Testumgebungen für die Erforschung von RL-Methoden macht.

MicroRTS ist eine vereinfachte Version von RTS-Spielen, die speziell für die Forschung entwickelt wurde [Huang et al., 2021]. Sie ist diskret und reduziert die visuelle und mechanische Komplexität von großen kommerziellen RTS-Spielen, zum Beispiel StarCraft II. Währenddessen behält sie die Kernschwierigkeiten, wie Ressourcenmanagement, Einheitenproduktion, Kampfmechaniken und taktische Positionierung, bei. MicroRTS eignet sich deshalb für die systematische Untersuchung von Lernverfahren. MicroRTS enthält weiterhin zahlreiche Trainingsherausforderungen, wie hohe kombinatorische Aktionsräume, verzögerte Belohnungen, Multi-Agent-Interaktionen und es erfordert auch robuster Strategien gegen unterschiedliche Gegner. Beispielsweise muss sich der Agent früh entscheiden, ob er gegen den aktuellen Gegner aggressiv oder defensiv spielen will, was sich auf die gesamte Spielstrategie auswirkt und eine langfristige Planung erfordert.

Ein zentrales Problem aus RL ist die Generalisierung gegenüber unterschiedlichen Gegnern und Strategien: Agenten lernen oft Strategien, die stark auf den Trainingsgegner oder eine spezifische Spielsituation zugeschnitten sind. Das führt zu Überanpassung und eingeschränkter Robustheit. SP, Fictitious Self-Play (FSP) und LT sind Ansätze, die dieses Problem adressieren. In SP trainiert der Agent, in dem er gegen sich selbst spielt, was zu einem kontinuierlichen Lernprozess führt, der keine zusätzlichen diversen oder starken Gegner erfordert. Allerdings ist oft das Training instabil, weil es zyklisch lernt. Um die Instabilitäten von reinem SP zu adressieren, wird in FSP nicht gegen den aktuellen Agenten sondern gegen ältere Versionen des Agenten gespielt, was ein zyklisches Lernen verhindert und in zwei Spieler Nullsummenspielen gegen das Nash-Gleichgewicht konvergiert [Heinrich et al., 2015]. LT erweitert FSP, indem der Agent gegen historische Gegner aus einer Liga trainiert, die aus verschiedenen Agententypen besteht, beispielsweise aus Policy-Snapshots früherer Trainingsstände sowie spezialisierten Agenten [Vinyals et al., 2019]. Ziel ist es, die Lernumgebung zu stabilisieren, in der Agenten gegen ein breites Spektrum an Strategien

antreten und dadurch robuster werden.

Trotz der theoretischen Vorteile ist die praktische Implementierung von LT in komplexen Umgebungen, wie MicroRTS, mit erheblichen Herausforderungen verbunden. Dazu gehören:

- die effiziente Verwaltung und Auswahl von Gegnern für das Training des Haupt-Agenten und für die spezialisierten Agenten,
- die Stabilisierung des Trainingsprozesses bei nicht-konstanten Gegnerverteilungen,
- der Umgang mit großen Aktionsräumen und spärlichen oder verzögerten Belohnungen und
- die Bewertung der tatsächlichen Generalisierungsfähigkeit der trainierten Agenten.

Darüberhinaus werden wir die für die unterschiedlichen Trainingsparadigmen die Lernkurve, die Strategiediversität und die Robustheit vergleichen und evaluieren.

Der in diesem Projekt eingesetzte Basis-Agent, wird in einem dreistufigen Verfahren trainiert, das BC und BC-Fine-Tuning (BC-FT) mit anschließendem Fine-Tuning durch PPO kombiniert: Die beiden BC-Phasen ermöglichen es dem Agenten, durch Imitation von Expertenverhalten schnell eine kompetente Basisstrategie zu erlernen [Torabi et al., 2018]. Dieser Ansatz des Trainings reduziert die anfängliche, oft ineffiziente Exploration in DRL [Torabi et al., 2018]. Das Lernen von Grund auf mit DRL-Methoden wie PPO kann sehr zeitintensiv sein [Martin and Jhala, 2021]. PPO ist eine bekannte on-policy Policy-Optimierungsmethode, die die Policy-Update-Größe über eine Clipped-Surrogat-Zielfunktion begrenzt [Schulman et al., 2017]. PPO verbessert weiterhin die Leistung des BC-Agenten durch Exploration und Anpassung an die Umgebung, wodurch sich der Agent über die Expertenstrategien hinaus anpassen und generalisieren kann [Martin and Jhala, 2021]. Dieser zweistufige Ansatz, bei dem eine mit BC vor-trainierte Policy durch einen DRL-Algorithmus verfeinert wird, hat sich als effektiv erwiesen, um die Trainingszeit zu verkürzen und die Gesamtleistung im Vergleich zu einem reinen DRL-Ansatz zu verbessern [Martin and Jhala, 2021].

2. Related Work

Game Playing Agents in MicroRTS Die dieser Arbeit zugrunde liegende (unveröffentlichte) Bachelorarbeit von Huft entwickelt einen wettbewerbsfähigen Agenten für MicroRTS und evaluiert eine hybride Trainingspipeline aus BC, BC-Fine-Tuning (BC-FT) und anschließendem PPO-Training [Huft, 2025]. Aufbauend auf der in der Introduction beschriebenen Grundidee wird die initiale

Policy aus Demonstrationen von Experten gelernt. In BC-FT wird dies mit selektierten Winning-Replays für die Stabilisierung weitergeführt und erst im dritten Schritt durch PPO weiter verfeinert. Dieses stufenweise Vorgehen reduziert die anfängliche Explorationslast und verbessert die Stabilität gegenüber einem reinen DRL-Start.

In den Ergebnissen ist ein deutlicher Leistungsanstieg über alle drei Trainingsphasen zu sehen (Tabelle 1): BC lernt größtenteils grundlegende Spielmechaniken, während BC-FT Generalisierung gegen mehrere Bots verbessert. Schließlich erreicht PPO hohe Winrates gegen viele der Built-in-Bots [Huft, 2025]. In den Ergebnissen sind Agenten, gegen die der Basis-Agent nicht gut abschneidet. Die Arbeit identifiziert zudem Einschränkungen, insbesondere den Fokus auf eine einzelne Karte, die begrenzte Gegnerdiversität und schlägt Erweiterungen mit Self-Play- bzw. Liga-Mechanismen vor. Genau an diesem Vorschlag setzt diese Arbeit an, indem der bestehende Basis-Agent und der initiale Code um SP und LT erweitert werden. Die Built-in-Bots von MicroRTS, gegen die unter anderem ausgewertet wird, sind RandomBiasedAI, WorkerRushAI, LightRushA und CoacAI.

Table 1. Winrates (%) for BC, BC-FT, and PPO reported in [Huft, 2025].

Bot	BC	BC-FT	PPO
WorkerRushAI	3	50	99
PassiveAI	98	100	100
LightRushAI	22	93	99
CoacAI	0	8	96
Mayari	0	5	95
RandomAI	52	95	99
RandomBiasedAI	18	68	86
rojo	0	34	84
mixedBot	4	44	59
izanagi	0	62	61
tiamat	11	78	84
droplet	0	31	59
guidedRojoA3N	4	42	75
naiveMCTS	0	5	3

MicroRTS-Py (Gym- μ RTS) Huang et al. stellen mit *Gym- μ RTS* bzw. *MicroRTS-Py* für unser Environment MicroRTS [Ontañón, 2013] eine effiziente Gym-kompatible Schnittstelle bereit [Huang et al., 2021]. MicroRTS-Py ist eine etablierte und weit verbreitete DRL-Baseline, die die Untersuchung von DRL-Ansätzen in vollständigen Echtzeitstrategiespiel-Szenarien ermöglicht, ohne dass dafür umfangreiche technische Infrastruktur, wie Hochleistungsrechner-Cluster, erforderlich ist [Huang et al., 2021]. Desweiteren wurde die *openai/baselines* [Dhariwal

et al., 2017] -Implementation von PPO adaptiert, um sie auf MicroRTS-Py anzupassen. Einen Großteil dieser Implementation setzen wir in LT ein. Sie wurde auch zuvor in der PPO Trainingsphasen des Basis-Agenten verwendet [Huft, 2025]. Zu openai/baselines wurden unter anderem *invalid action masking* und geeignete Aktions-/Beobachtungsrepräsentationen (z. B.: GridNet [Han et al., 2019]) sowie architektonische Entscheidungen (wie IMPALA-CNN (eine tiefe Convolutional-Neural-Network-Struktur mit residual blocks zur Verarbeitung gitterbasierter Zustandsrepräsentationen in RL) [Espeholt et al., 2018]) hinzugefügt [Huang et al., 2021], die abgeändert auch in dem LT-Code noch genutzt werden. Es wurde in dem Paper überlegt, *Unit Action Simulation* (UAS) statt GridNet zu verwenden. UAS iteriert in jedem Schritt über alle Einheiten und gibt für jede Einheit die möglichen Aktionen zurück, was den Aktionsraum deutlich reduziert. GridNet hingegen sagt für jede Zelle auf der Karte eine Aktion voraus. Das führt GridNet mit seinem großen Aktionsraum in einem Schritt aus, aber das macht das Training simplistischer [Huang et al., 2021, Han et al., 2019]. Gegen die Built-in-Bots von MicroRTS, die auf heuristischen Regeln basieren, erreicht die PPO-Implementierung mit *invalid action masking* und IMPALA-CNN im Durchschnitt eine Winrate von 91% mit UAS und 84% mit GridNet. Letztendlich wurde entschieden, sich auf GridNet zu fokussieren, da die Komplexität geringer ist und die Kompatibilität SP (bzw. LT) erleichtert [Huang et al., 2021]. SP wurde in der Arbeit auch evaluiert. Allerdings gab es in dem Code ein Bug, der wahrscheinlich die Ergebnisse für SP stark beeinflusst hat [Goodfriend, 2024].

RAISocketAI [Goodfriend, 2024] RAISocketAI ist der erste DRL-Agent, der im Jahr 2023 die IEEE MicroRTS Competition gewinnen konnte. RaiSocketAI beinhaltet eine Neu-Implementation von MicroRTS-Py, die in unserem Code verwendet wird. Unter anderem ist die Datengenerierung über Replays ist eine wichtige Grundlage für BC [Huft, 2025] und es wurde ein Bug in der alten Implementation behoben, der nicht besitzte Recourcen als besitzte Recourcen markierte, wenn der Agent Spieler 2 ist. Das führte laut Goodfriend wahrscheinlich dazu, dass SP in der vorherigen Implementation von MicroRTS-Py keine Verbesserungen brachte: Die RAISocketAI-Implementierung von FSP hat gegen die gleichen Built-in-Bots von MicroRTS wie in MicroRTS-Py eine durchschnittliche Winrate von 91% erreicht, was deutlich höher ist als 84% in der vorherigen Implementation. Allerdings erreichte FSP gegen CoacAI nur eine 20% Winrate, während die besten vorherigen Implementationen fast immer CoacAI besiegen

konnten. Mit finetuning CoacAI und Mayari wurde die durchschnittliche Winrate gegen die Built-in-Bots auf 98% erhöht [Goodfriend, 2024]. LT wurde in der Arbeit nicht implementiert oder evaluiert.

BeClone [Martin and Jhala, 2021] *BeClone* zeigte die Effektivität von BC mit DRL-finetuning in MicroRTS-Py mit einer zweiphasigen Trainingsstrategie für RTS-Agenten: Zuerst wird eine initiale Policy per BC von den Experten gelernt, anschließend erfolgt ein RL-Finetuning mittels *Advantage Actor-Critic* (A2C).

AlphaStar [Vinyals et al., 2019] AlphaStar demonstriert, dass LT in komplexen RTS-Domänen entscheidend zur Robustheit beitragen kann. Der Agent wurde mit *Imitation Learning* aus menschlichen Replays vortrainiert. Das haben wir in dieser Arbeit mit den BC-Phasen und der PPO-Phase ersetzt. Anschließend wird der Agent, wie auch in dieser Arbeit, in einer Liga aus unterschiedlichen Agententypen (Main-Agent, Main-Exploiters, League-Exploiters und historische Snapshots) weitertrainiert, um Überanpassung und Vergessen zu reduzieren und Diversität zu fördern. Obwohl MicroRTS und StarCraft II in Komplexität und Repräsentation stark differieren, zeigt sich AlphaStar als ein etabliertes Referenzdesign für die Stabilisierung und Generalisierung von SP, durch ligaartige Matchmaking-Mechanismen und die Einbeziehung von spezialisierten Exploitern [Vinyals et al., 2019]. Wie auch in AlphaStar, werden in dieser Arbeit Main-Exploiters trainiert, indem der Basis-Agent gegen alle historischen Versionen der Main-Agenten in der Liga trainiert wird, um Schwachstellen zu identifizieren und auszunutzen [Vinyals et al., 2019]. Gleichzeitig trainieren League-Exploiters gegen alle *Historischen Agenten* [Vinyals et al., 2019]. Der Main-Agent trainiert mit *prioritized FSP* (PFSP) gegen historische Versionen von sich selbst, Main-Exploiters und League-Exploiters [Vinyals et al., 2019]. Die Priorisierung der historischen Gegneragenten erfolgt durch die Winrate des Main-Agenten gegen die jeweiligen historischen Agenten [Vinyals et al., 2019]. Erst, wenn ein Exploiter eine große Winrate gegenüber allen Gegnern aufweist oder wenn eine bestimmte maximale Schritt-Grenze erreicht ist, wird er in die Liga als Historical (ein historischer Agent) aufgenommen [Vinyals et al., 2019]. In AlphaStar kommt ein *Actor-Critic-Verfahren* mit einer Off-Policy-Korrekturmethode namens *V-trace* für das Policy-Update zum Einsatz [Espeholt et al., 2018, Vinyals et al., 2019]. Dieser Ansatz ist durch die Robustheit gegenüber Off-policy-Daten und der typischerweise besseren Skalierbarkeit in großen verteilten Setups besser für LT geeignet als PPO. Jedoch ist die Implementierung von PPO in MicroRTS-Py deutlich einfacher und schneller realisierbar, weshalb PPO eine praktikable Wahl für

- **type:** Aktionstyp (Range: [0, 5], Nop, Move, Harvest, Return, Produce, Attack).
- $d_{\text{move}}, d_{\text{harvest}}, d_{\text{return}}, d_{\text{produce}}$: Richtungen (Range: [0, 3], North, East, South, West) für die jeweiligen Aktionen.
- **produceType:** Typ der zu produzierenden Unit je nach produzierenden Unit. (Range: [0, 6],).
- **relAttackPos:** Relative Angriffsposition. (Range: [0, 48], (0: Resource), 1: Base, 2: Barracks, 3: Worker, 4: Light, 5: Heavy, 6: Ranged)

0	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23		25	26	27
28	29	30	31	32	33	34
35	36	37	38	39	40	41
42	43	44	45	46	47	48

Figure 2. Relative Angriffsposition.

In unserem Agenten wird der Unit-Action-Space (ohne die Positionsdimension) linearisiert und mittels One-Hot-Kodierung dargestellt. Die Position ist implizit durch die lineare Gitterindizierung der GridNet-Ausgabe kodiert [Huang et al., 2021]. Gegeben eine Karte der Größe $h \times w$, dann besteht der Aktionsraum aus $h \times w \times 78$. In unserer 16×16 Karte würde der Aktionsraum für ein Environment in der Darstellung von GridNet also $16 \times 16 \times 78 = 19968$ Aktionen umfassen, von denen jedoch viele ungültig sein können und durch invalid action masking maskiert werden [Huang et al., 2021]. Je nach Aktionstyp sind nur Teile des Aktionsvektors relevant (z.B.: wenn der Aktionstyp Move ist, dann sind nur die Komponenten pos, type und d_{move} relevant, die anderen Komponenten werden ignoriert).

3.2. Training Pipeline

Die in dieser Arbeit erweiterte Trainingspipeline besteht aus vier Phasen: BC, BC-FT, PPO und LT (oder FSP/SP). Die ersten drei Phasen wurden bereits in der vorherigen Arbeit implementiert [Huft, 2025] und werden hier nur kurz beschrieben. In Bc werden Demonstrationen von Experten (hier die Built-in-Bots CoacAI und Mayari) genutzt, um eine initiale Policy zu trainieren, die grundlegende Spielmechaniken erlernt hat [Huft, 2025]. BC-FT stabilisiert die Policy weiter und erreicht kompetente Basisstrategien, indem mit ausgewählten Winning-Replays der Experten weiter trainiert wird [Huft, 2025]. Die beiden BC-Phasen reduzieren die anfängliche, oft ineffiziente Exploration in DRL wie PPO, das sehr zeitintensiv sein kann, wenn es von Grund auf trainiert wird [Torabi et al., 2018, Martin and Jhala, 2021]. Insbesondere in MicroRTS, wo die Belohnungen anfänglich besonders spärlich und verzögert sein können, durch die hohe kompetenz der Gegner, die der Agent zu Beginn des Trainings hat, und durch die Komplexität der Umgebung, kann ein reines DRL-Training ohne Vortraining sehr ineffizient sein [Martin and Jhala, 2021]. Die on-policy Policy-Optimierungsmethode PPO, begrenzt die Policy-Update-Größe über eine Clipped-Surrogat-Zielfunktion und ermöglicht dadurch ein stabiles Training [Schulman et al., 2017]. PPO wird in dieser Arbeit eingesetzt, um die Leistung des BC-Agenten weiter durch zusätzliche Exploration kann sich der Agent über die Experten hinaus Anpassen und besser generalisieren [Martin and Jhala, 2021]. Typischerweise wird PPO als Actor-Critic-Verfahren dargestellt, in dem der Actor die Policy repräsentiert und der Critic die Value-Funktion schätzt [Schulman et al., 2017]. Aus den Trajectories, die in einem Rollout gesammelt wurden werden Advantage-Schätzungen berechnet. Diese werden als Signal für die Policy-Updates verwendet. In der Implementierung wird die Loss funktion, die aus der negativen PPO-Zielfunktion besteht mit einem Gradientenabstieg minimiert, was äquivalent zur Maximierung der PPO-Zielfunktion ist [Schulman et al., 2017]. Die PPO-Zielfunktion besteht aus drei zentralen Termen: einem Policy-Term, der die erwartete Belohnung unter der Begrenzung der Policy-Updates durch die Clipping-Funktion maximiert, einem Value-Term für die Minimierung des Fehlers in der Value-Schätzung und einen Entropie-Term, der durch Maximierung der Aktionsentropie die Exploration fördert [Schulman et al., 2017]. Mit SP/FSP/LT streben wir an den Basis-Agenten weiter zu generalisieren, indem PPO nicht ausschließlich gegen die Built-in-Bots von MicroRTS trainieren, gegen die der Basis-Agent bereits eine hohe Winrate erreicht, sondern auch gegen gleich starke oder stärkere Gegner. Viele Gegner auf diesem Leistungsniveau sind jedoch

nicht für das Trainieren geeignet, da sie sehr langsam sind und dadurch die Sammlung der Rollouts verlangsamen. Dazu kommt, dass sich die Generalisierungsfähigkeit des Agenten mit wenigen neuen Gegnern im Training nur begrenzt verbessert, wenn die Trainingsumgebung nicht eine ausreichende Vielfalt an Gegnern geboten wird. SP/FSP/LT adressieren beide Probleme, indem sie eine große Anzahl von Gegnern, die sich im Laufe des Trainings weiterentwickeln und eine hohe Winrate gegen historische Versionen des aktuellen Agenten aufweisen, bereitstellen. In SP trainiert der Agent gegen seine eigene aktuelle Version, was zu einem kontinuierlichen Lernprozess mit einem sich ko-evolvierenden Gegner führt [Silver et al., 2018]. Allerdings ist oft das Training instabil, weil es zyklisch lernt. Ein Beispiel: Spielt der Agentzustand B gut gegen A, C gut gegen B und A gut gegen C, könnte der Agent in einem Zyklus gefangen sein, in dem er immer nur den Agentenzustand wieder erlernt, der gegen sich selbst gut ist, aber nicht gegen andere Gegner.

Um die Instabilitäten von reinem SP zu adressieren, wird in FSP nicht ausschließlich gegen die aktuelle Policy, sondern gegen eine gleichverteilten Menge historischer Policies des Agenten trainiert, sodass der lernende Agent eine Best Response gegen die gemittelte Gegnerstrategie lernt, was ein zyklisches Lernen reduziert. Unter Standardannahmen kann FSP in Zwei-Spieler-Nullsummenspielen (bezogen auf die Durchschnittsstrategien) gegen ein Nash-Gleichgewicht konvergieren. Heinrich et al. zeigen dies zudem empirisch in Poker-Experimenten [Heinrich et al., 2015]. LT verwendet eine erweiterte Variante von FSP, die Prioritized fictitious self-play (PFSP) genannt wird [Vinyals et al., 2019]. PFSP erweitert FSP, indem es nicht die Gegner gleichverteilt auswählt, sondern Gegner auswählt, gegen die der Agent effizient trainieren kann. Die Auswahlwahrscheinlichkeit basiert dabei auf der Winrate des lernenden Agenten gegen den potenziellen Gegner, wobei die genaue Priorisierungsfunktion für unterschiedliche Agententypen unterschiedlich sein kann [Vinyals et al., 2019]. In AlphaStar werden für die Main-Agenten Gegner mit mittlerer Schwierigkeit (Winrate nahe an 50%) höher priorisiert und für Exploiter werden starke Gegner (Winrate nahe an 0%) priorisiert [Vinyals et al., 2019]. Da in dieser Arbeit alle Agententypen mit dem Basis-Agent initialisiert werden und sich die Winrates in Starcraft II und MicroRTS sich unterscheiden, werden angepasste Priorisierungsfunktionen verwendet, die von den in AlphaStar eingesetzten Funktionen abweichen [Vinyals et al., 2019]. Ein wesentlicher Unterschied ist, dass kaum informative Gradienten bzw. Lernsignale entstehen, wenn gegen einen Gegner, gegen den der Basis-Agent eine Winrate nahe 0% hat, trainiert

wird.

Um diese Gegner zu vermeiden wurde die folgende Priorisierungsfunktion für den Main-Exploiter verwendet: Wie in dem Bild zu sehen wird eine Winrate von 0% über eine Winrate von 100% priorisiert, da aus einer Winrate von 0% mehr gelernt werden kann als aus einer Winrate von 100%, die meistens heißt, dass der Agent den Gegner bereits gut besiegt. Was nicht in dem Bild dargestellt ist, ist dass alle Agenten eine Wahrscheinlichkeit von über 0% haben, ausgewählt zu werden, damit kein Gegner der initial zufälligerweise immer verloren hat und damit der lernende Agent gegen diesen Gegner eine Winrate von 100% hat, komplett ignoriert wird, da es immer noch gut möglich ist, dass der Agent gegen diesen Gegner gewinnt und dadurch lernt.

ScalarEncoder

$$\text{ScalarEncoder} : \mathbb{R}^{d_{in}} \rightarrow \mathbb{R}^{d_{out}},$$

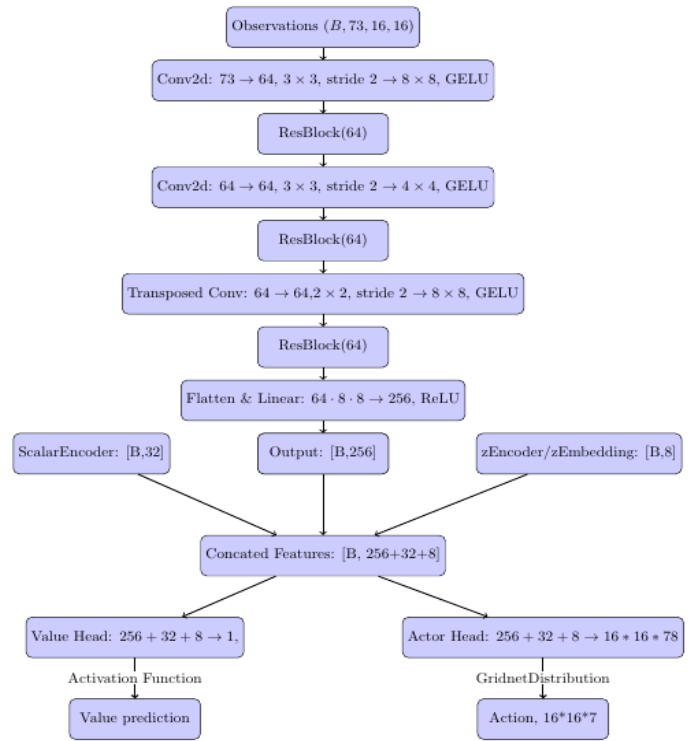


Figure 3. The Network Architecture

4. Experimente

- **Generalization to random opponents:**
- **Map and observability constraints:**
- **Replay and training bottlenecks:**

Table 2. Training parameters for PPO training.

Parameter	Value
Initial learning rate (Adam)	2.5×10^{-5}
Rollout steps per environment	512
Minibatch Size	3072
Max episode length	2,000 timesteps
Discount factor γ	1
Value function coefficient β	0.01
Entropy coefficient η	0.005
Gradient norm threshold ω	0.5
KL regularization coefficient λ_{KL}	0.4
GAE parameter λ	0.95

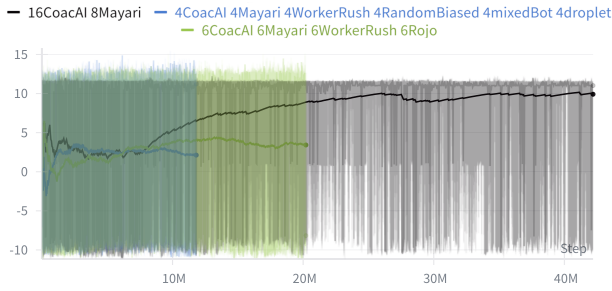


Figure 4. Episode rewards (y-axis) as a function of training timesteps (x-axis) on the BC-FT agent.

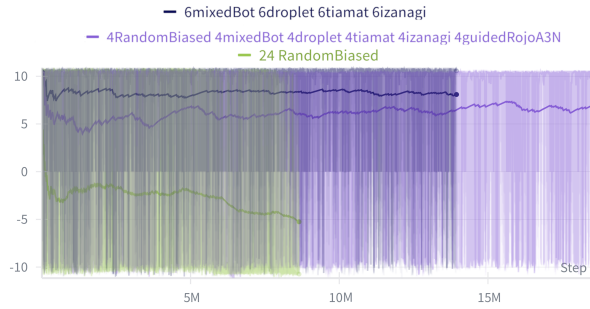


Figure 5. Episode rewards (y-axis) as a function of training timesteps (x-axis) on the final agent.

• Exploitability:

5. Conclusion

References

- [Dhariwal et al., 2017] Dhariwal, P., Hesse, C., Klimov, O., Nichol, A., Plappert, M., Radford, A., Schulman, J., Sidor, S., Wu, Y., and Zhokhov, P. (2017). Openai baselines. <https://github.com/openai/baselines>. 2
- [Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). Impala: scalable distributed deep-rl with importance weighted actor-learner architectures. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 1406–1415. 3
- [Goodfriend, 2024] Goodfriend, S. (2024). Raisocketai: The first deep reinforcement learning agent to win the iee microrts competition. In *2024 IEEE Conference on Games (CoG)*, pages 1–8. 3, 4
- [Han et al., 2019] Han, L., Sun, P., Du, Y., Xiong, J., Wang, Q., Sun, X., Liu, H., and Zhang, T. (2019). Grid-wise control for multi-agent reinforcement learning in video game AI. In Chaudhuri, K. and Salakhutdinov, R., editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2576–2585. PMLR. 3, 4
- [Heinrich et al., 2015] Heinrich, J., Lanctot, M., and Silver, D. (2015). Fictitious self-play in extensive-form games. In Bach, F. and Blei, D., editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 805–813. 1, 6
- [Huang et al., 2021] Huang, S., Ontańón, S., Bamford, C., and Grela, L. (2021). Gym-microrts: Toward affordable full game real-time strategy games research with deep reinforcement learning. *CoRR*, abs/2105.13807. 1, 2, 3, 4, 5
- [Huft, 2025] Huft, M. (2025). Game playing agents in microrts. 2, 3, 4, 5
- [Martin and Jhala, 2021] Martin, D. and Jhala, A. (2021). Beclone: Behavior cloning with inference for real-time strategy games. In *Joint Proceedings of the AIIDE 2021 Workshops*. 2, 3, 5
- [Ontańón, 2013] Ontańón, S. (2013). The combinatorial multi-armed bandit problem and its application to real-time strategy games. In *Proceedings of the Ninth Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, volume 9, pages 58–64. AAAI. 2
- [Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *CoRR*, abs/1707.06347. 2, 5
- [Silver et al., 2018] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362:1140–1144. 6
- [Sutton and Barto, 1998] Sutton, R. S. and Barto, A. G. (1998). *Reinforcement Learning: An Introduction*. The MIT Press. 4
- [Torabi et al., 2018] Torabi, F., Warnell, G., and Stone, P. (2018). Behavioral cloning from observation. *arXiv preprint arXiv:1805.01954*. 2, 5

[Vinyals et al., 2019] Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., Oh, J., Horgan, D., Kroiss, M., Danihelka, I., Huang, A., Sifre, L., Cai, T., Agapiou, J. P., Jaderberg, M., Vezhnevets, A. S., Leblond, R., Pohlen, T., Dalibard, V., Budden, D., Sulsky, Y., Molloy, J., Paine, T. L., Gulcehre, C., Wang, Z., Pfaff, T., Wu, Y., Ring, R., Yogatama, D., Wünsch, D., McKinney, K., Smith, O., Schaul, T., Lillicrap, T., Kavukcuoglu, K., Hassabis, D., Apps, C., and Silver, D. (2019). Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354. [1](#), [3](#), [4](#), [6](#)

Erklärung

Hiermit versichere ich, dass ich die Bachelorarbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Düsseldorf, den XXXXXXXXXXXX

Name!!